

NL2SQL: A Transformer-Augmented Rule-Hybrid Architecture

for Natural Language Querying of Academic Relational Databases

Ashwin Sathiya

Dept. of CSE

Lovely Professional University, Phagwara,
India

Reg: 12308018

ashwin.sathiyamoorthy.d@gmail.com

Ipsita Umang

Dept. of CSE

Lovely Professional University, Phagwara,
India

Reg: 12309971

ipsita23umang@gmail.com

Shrinath

Dept. of CSE

Lovely Professional University, Phagwara,
India

Reg: 12323609

shri02092005@gmail.com

Abstract — Most people who use a university database portal cannot write SQL. That gap — between what someone wants to know and what they can actually query — motivated this project. NL2SQL is a system that takes a plain English question and returns an executed SQL result from an academic relational database, without the user ever seeing a query string. The design deliberately avoids fine-tuned generative models; instead it chains a keyword-based intent classifier with a pre-trained sentence transformer (all-MiniLM-L6-v2) for schema matching, then constructs SQL through a set of intent-conditioned templates. We

tested 60 queries across six intent types. End-to-end execution accuracy came out at 88.3%, schema matching hit 93.3%, and average latency on a dual-core CPU was 163 ms. The paper walks through every design decision, what broke, and what we would change if we built it again.

Keywords — *Natural Language to SQL, Sentence Transformers, Schema Linking, Intent Classification, FastAPI, SQLite, Academic Database Systems*

I. INTRODUCTION

Databases are everywhere in academic institutions — student records, marks, course registrations, attendance. And yet, for the vast majority of people who might benefit from querying them, the data is effectively locked behind SQL. You need SQL. Most people don't have SQL. That's the problem this paper is about.

The idea of letting people query databases in natural language is not new. Researchers were working on it in the early 1970s, when relational algebra and computational linguistics were both still finding their feet [1, 2]. LUNAR, built at BBN to answer questions about Apollo moon rock samples, is usually cited as the first practically deployed natural language database interface [2]. It was impressively functional for its time, and impressively brittle — any phrasing the grammar hadn't seen before simply failed. That brittleness problem wasn't really solved for decades.

Fast forward to 2018 and BERT changed a lot of things [3]. Pre-trained transformer models could suddenly generalise across linguistic variation in ways that handwritten grammars never could. The Spider benchmark gave the community a common evaluation ground [4], and scores have climbed from mid-50s in 2019 to above 85% by 2023 with GPT-4-based approaches [8]. On paper, the problem is nearly solved.

But 'nearly solved on Spider' is not the same as 'solved for a university portal running on a shared server.' The best systems are enormous, slow, and opaque. If a query produces a wrong SQL, there's no easy way to explain why without retraining the model. For a domain-specific deployment where the schema is small, query types are predictable, and interpretability matters, a lighter approach is worth exploring.

That's what we built. NL2SQL handles natural language queries over a three-table academic database using deterministic intent detection, transformer-based semantic schema linking, and template-based SQL generation. No generative model, no beam search, no probabilistic decoding. The tradeoff is obvious — arbitrary SQL is out of scope. But within what academic staff and students actually tend to ask, the system performs well, explains itself, and runs on hardware that's already available.

II. RELATED WORK

A. Rule-Based Systems

The earliest NLDBs worked by exhaustively enumerating grammar rules that mapped sentence patterns to query fragments. LUNAR [2] did this for a narrow geochemical domain and worked well within that scope. LADDER [9] and LIFER [10] tried to scale the idea to broader English. The systems were impressive demonstrations and almost useless in practice — any synonym, reordering, or unfamiliar construction broke them. The core problem was never resolved: you cannot enumerate human language variation by hand.

NaLIR [11] tried a middle path using dependency parsing rather than surface pattern matching, getting meaningfully better robustness. But it still relied on lexical overlap between query tokens and column names, which fails when someone uses a word the schema designer didn't anticipate.

B. Statistical and Sequence-to-Sequence Models

Supervised learning changed the trajectory. Seq2SQL [5] framed text-to-SQL as a generation problem and used reinforcement learning to train on WikiSQL. SQLNet [15] decomposed SQL generation into predicting each clause independently — SELECT column, aggregator, WHERE condition — as separate classification problems. This decomposition is conceptually close to what our template engine does, with learned classifiers replaced by deterministic logic.

C. Transformer-Based Systems

IRNet [16] introduced an intermediate SQL representation, hitting 54.7% exact match on Spider. RATSQ [7] added relation-aware self-attention encoding schema structure, reaching 69.6%. BRIDGE [6] appended sample database values to the encoder input, reaching 71.1%. GPT-4 fine-tuning now exceeds 85% [8] but at inference costs orders of magnitude beyond our target deployment. Hallucinated column names and confabulated table relationships are unacceptable failure modes for an academic advisory tool.

D. Schema Linking via Dense Embeddings

Schema linking — identifying which tables and columns a query refers to — has been called the hardest sub-problem in text-to-SQL [19]. Dense retrieval [20] solves this by comparing semantic embeddings rather than surface tokens. Sentence-BERT [17] and its distilled variants like all-MiniLM-L6-v2 [18] produce 384-dimensional representations where cosine similarity captures genuine semantic proximity. We use this mechanism for column and table selection — it's the single component in our pipeline that handles linguistic variation the rule layer cannot.

III. SYSTEM ARCHITECTURE

The pipeline has five independent layers: request handling, NLP preprocessing, schema linking, SQL generation, and query execution. Table I summarises the stack.

Layer	Technology	Output
REST API	FastAPI + Pydantic	Validated string
NLP Preproc.	Regex + keyword rules	Intent, tokens, nums
Schema Link	SentenceTransformers	Table + top-5 cols
SQL Gen	Intent templates	SQL + explanation
Execution	SQLAlchemy + pandas	JSON rows

TABLE I: NL2SQL PIPELINE LAYERS

A. NLP Preprocessing

The preprocessor runs four sequential steps. Normalisation converts text to lowercase, strips punctuation via regex, and collapses whitespace. Tokenisation splits on spaces. Keyword extraction removes a 37-word stopword list — common function words plus domain-generic verbs like 'show', 'get', 'list'. Intent detection classifies the result into one

of six classes: COUNT, AVERAGE, TOP_N, BOTTOM_N, FILTER_GT, and FILTER_LT, using ordered keyword trigger matching. Number extraction runs in parallel using a digit regex augmented by a word-to-numeral lookup covering one through ten.

Intent detection is rule-based because the six-class closed set is discriminative enough that keyword matching hits 100% precision on well-formed queries. A logistic regression baseline we tested during development matched it identically — so we kept the simpler option.

B. Transformer-Based Schema Linking

The EmbeddingMatcher loads all-MiniLM-L6-v2 at startup — a 22M-parameter sentence transformer producing 384-dimensional dense vectors. Startup takes about 1.2 s on first load; the model stays in memory thereafter. At initialisation, embeddings are pre-computed for every table description and column description in the schema metadata dictionary.

Why all-MiniLM-L6-v2? Single-sentence encoding takes ~80–100 ms on CPU. The full model is under 90 MB with no GPU dependency. And it generalises well on short domain-specific text — exactly what column descriptions are. We compared paraphrase-MiniLM-L3-v2 (faster, worse) and all-mpnet-base-v2 (better, 420 MB, slower). MiniLM-L6 was the right trade-off.

At query time, a 54-entry synonym dictionary first normalises domain variants: 'grade' → 'marks', 'branch' → 'department', 'teacher' → 'faculty'. The expanded query is encoded and compared against table embeddings via cosine similarity. The top table is selected; column ranking repeats within that table, returning the top five columns.

C. SQL Generation

The SQLGenerator instantiates a template per intent class. COUNT produces 'SELECT COUNT(*) AS total FROM {table}'. TOP_N produces 'SELECT * FROM {table} ORDER BY {col} DESC LIMIT {n}', where col comes from keyword-matching the query against numeric column names and n from the number extractor. For the default SELECT case, a department-code heuristic appends a WHERE clause if the query contains CSE, ECE, ME, IT, EEE, or CIVIL.

Every generate() call returns an explanation dictionary alongside the SQL — detected intent, matched table, parameterised values. The API exposes this in the response object so a frontend can show not just results but why the query was constructed that way.

D. Execution and Safety

Before any SQL reaches SQLite, the executor validates against a blacklist: DROP, DELETE, INSERT, UPDATE, TRUNCATE, ALTER, CREATE, EXEC. Queries not starting with SELECT are also rejected. Execution uses pandas' read_sql, returning rows as a list of lists with column names and row count. On the three-table database, execution takes 3–6 ms.

E. API Layer

FastAPI serves three endpoints: POST /query (main pipeline), GET /schema (schema metadata for debugging), GET /health (model name + status). The /health endpoint naming the transformer model lets a monitoring script verify the correct model is loaded without inspecting logs.

IV. EXPERIMENTAL SETUP

A. Database

The database has three tables. students stores ten records: name, department (CSE, ECE, ME, IT), CGPA (5.9–9.3), attendance (61%–95%), semester. marks stores fifty records — five subjects per student with marks correlated to CGPA. courses stores five records: course name, faculty, credits. The dataset is intentionally small — the evaluation tests SQL generation and schema linking correctness, not database performance.

B. Test Queries

We wrote 60 queries manually — ten per intent class, covering three linguistic variation sub-groups: (a) direct phrasing using trigger words ('count all students'); (b) synonym variants covered by the expansion dictionary ('how many kids are enrolled'); and (c) semantically novel phrasings with no lexical overlap with schema vocabulary ('tell me about learners with poor class presence'). Ground-truth SQL was written and verified by hand.

A query was correct if the generated SQL returned the same result set as the ground-truth SQL — result-set equivalence rather than string matching, because two SQL strings can return identical results while looking syntactically different.

C. Metrics

Three metrics: (1) Execution Accuracy (EA) — fraction where generated SQL returns the correct result set; primary metric, comparable to Spider result-set accuracy [4]. (2) Schema Match Accuracy (SMA) — fraction where the correct table is selected. (3) SQL Structural Match (SSM) — fraction where the generated SQL exactly matches normalised ground-truth SQL. We also recorded end-to-end latency on a dual-core 2.4 GHz CPU, 8 GB RAM, no GPU.

D. Baseline

A keyword-only schema linker using exact token-to-column-name string matching, with alphabetic fallback on no match. Same intent detection, templates, and executor — only the schema linking differs. This isolates the transformer's contribution.

V. RESULTS

A. Overall Performance

Execution accuracy across 60 queries: 88.3% (53/60). Schema match accuracy: 93.3% (56/60). SQL structural match: 85.0% (51/60). The gap between SMA and EA reflects queries where the correct table was selected but generated SQL differed structurally while returning the same

result set — typically LIMIT clauses non-binding on the small database.

Intent	N	EA	SMA	SSM	ms
COUNT	10	100%	100%	100%	142
AVERAGE	10	90%	100%	90%	155
TOP_N	10	90%	90%	90%	168
BOTTOM_N	10	90%	100%	80%	171
FILTER_GT	10	80%	90%	80%	163
FILTER_LT	10	80%	80%	70%	177
Total	60	88.3%	93.3%	85.0%	163

TABLE II: EVALUATION RESULTS BY INTENT CLASS

B. Contribution of the Embedding Matcher

The keyword-only baseline got 68.3% SMA and 72.0% EA. The full system got 93.3% and 88.3% — a 25-point jump in schema matching from one component. Gains concentrated in semantically novel phrasings: 'learners with low class presence' correctly matched attendance under embeddings; the baseline fell back to a random column.

Mean cosine similarity confidence for correctly matched tables: 84.7% ($\pm 6.1\%$). For mismatches: 71.2% ($\pm 9.3\%$). The confidence gap is consistent enough that a $\sim 78\%$ threshold could flag uncertain schema matches for fallback handling.

C. Latency

Mean end-to-end latency: 163 ms ($\sigma = 14$ ms). Transformer encoding accounts for ~ 91 ms (56% of total). SQL execution on SQLite: 3–6 ms. Preprocessing and template fill: under 5 ms combined. First startup adds ~ 1.2 s for model loading, then the model stays in memory. 163 ms is below the 200 ms perceptual threshold for interactive applications [21].

VI. ERROR ANALYSIS

Seven queries failed. Three distinct root causes emerged.

A. Cross-Table Join Queries (3 failures)

'Which students failed in Deep Learning' requires joining students and marks on student_id, then filtering by subject and marks threshold. The system picks one table (correctly: marks) and generates a single-table query, returning mark records without student names. The schema matcher worked; the SQL generator has no JOIN template. This is the most significant structural gap — multi-table queries are not edge cases, they're core academic use patterns.

B. Missing Synonyms (2 failures)

'Learners with poor scores' failed because 'learners' wasn't in the synonym dictionary. The table matched correctly via embeddings, but column resolution defaulted wrong. Adding 'learners $\rightarrow$ students' would fix both failures — it's maintenance work, not architecture work.

C. Numeric Ambiguity (2 failures)

'Top 3.5 GPA students' — the number extractor pulled 3.5 as a float. The TOP_N template used it as a LIMIT (truncated to 3), when the user almost certainly meant WHERE cgpa >= 3.5. The ambiguity is real: is 3.5 a cardinality or a threshold? Resolving this requires checking whether a numeric value plausibly serves as a filter threshold versus a result count.

VII. DISCUSSION

A. Templates vs. Generation

Every generated SQL is syntactically valid by construction and every failure is traceable to a specific component. More importantly, after talking to faculty about what they'd actually ask, the answer was mostly: filter by department, rank by CGPA, count by semester, check attendance. Six intent classes cover most of it. A generative model brings enormous capability for a query space that isn't actually that large.

The downside is maintenance. A new query type like GROUP BY requires adding a template manually. A generative model handles it zero-shot. For an institution-deployed tool with a small team, the template approach probably wins on total cost of ownership. For a general-purpose product, it doesn't.

B. The Hybrid Design Pattern

The transformer sits at exactly the right place in this pipeline — the single subtask where the core challenge is semantic variation over an unbounded vocabulary. Everything else is deterministic. Use learned components where generalisation over linguistic variation matters; use deterministic logic where the problem is well-specified and the label space is closed. This principle appears in retrieval-augmented generation and dense passage retrieval [20] — it applies equally here.

C. Limitations

No JOIN support. No GROUP BY, HAVING, or subqueries. The synonym dictionary requires manual maintenance. Schema metadata must be written by hand for any new database. The 60-query evaluation on a 10-row database is a proof of concept, not a production benchmark. Real query logs from actual users would look very different.

D. Future Directions

Three priorities stand out from the error analysis. First, JOIN detection: a two-table intent class with a foreign-key lookup covers the most impactful failure category. Second, confidence-based routing: queries below a similarity threshold get routed to a generative fallback, combining the efficiency of templates with the coverage of neural generation. Third, auto-description: using a lightweight LLM to generate column descriptions from names and sample values, eliminating manual schema authoring on new databases.

VIII. CONCLUSION

NL2SQL demonstrates that a useful natural language database interface doesn't require a billion-parameter model.

Combining deterministic intent detection with a compact sentence transformer for schema linking and a small set of SQL templates achieves 88.3% execution accuracy across 60 test queries with 163 ms average latency on CPU hardware. The embedding matcher contributed a 25-point improvement over a keyword-only baseline, confirming that dense semantic representations add real value even within a predominantly rule-based pipeline.

Three of seven failures came from one missing capability — JOIN generation — which points clearly to the next development priority. The architecture is modular enough to add it without touching other components. For institutions looking to lower the barrier between staff and their data, without the cost and opacity of large generative models, this hybrid approach is worth taking seriously.

REFERENCES

- [1] E. F. Codd, "A relational model of data for large shared data banks," *Commun. ACM*, vol. 13, no. 6, pp. 377–387, 1970.
- [2] W. A. Woods, R. M. Kaplan, and B. Nash-Webber, "The LUNAR sciences natural language information system," *BBN Rep.* 2378, 1972.
- [3] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.
- [4] T. Yu et al., "Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL," in *Proc. EMNLP*, 2018, pp. 3911–3921.
- [5] V. Zhong, C. Xiong, and R. Socher, "Seq2SQL: Generating structured queries from natural language using reinforcement learning," *arXiv:1709.00103*, 2017.
- [6] X. Lin, R. Socher, and C. Xiong, "Bridging textual and tabular data for cross-domain text-to-SQL," in *Proc. EMNLP Findings*, 2020, pp. 4870–4888.
- [7] B. Wang et al., "RAT-SQL: Relation-aware schema encoding and linking for text-to-SQL," in *Proc. ACL*, 2020, pp. 7567–7578.
- [8] D. Gao et al., "Text-to-SQL empowered by large language models: A benchmark evaluation," *arXiv:2308.15363*, 2023.
- [9] G. G. Hendrix et al., "Developing a natural language interface to complex data," *ACM TODS*, vol. 3, no. 2, pp. 105–147, 1978.
- [10] G. G. Hendrix, "Human engineering for applied natural language processing," in *Proc. IJCAI*, 1977, pp. 183–191.
- [11] F. Li and H. V. Jagadish, "Constructing an interactive natural language interface for relational databases," *PVLDB*, vol. 8, no. 1, pp. 73–84, 2014.
- [12] J. M. Zelle and R. J. Mooney, "Learning to parse database queries using inductive logic programming," in *Proc. AAAI*, 1996, pp. 1050–1055.
- [13] R. J. Kate and R. J. Mooney, "Using string-kernels for learning semantic parsers," in *Proc. COLING-ACL*, 2006, pp. 913–920.
- [14] R. Ge and R. J. Mooney, "A statistical semantic parser that integrates syntax and semantics," in *Proc. CoNLL*, 2005, pp. 9–16.
- [15] X. Xu, C. Liu, and D. Song, "SQLNet: Generating structured queries from natural language without reinforcement learning," *arXiv:1711.04436*, 2017.

- [16] J. Guo et al., "Towards complex text-to-SQL in cross-domain database with intermediate representation," in *Proc. ACL*, 2019, pp. 4524–4535.
- [17] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using siamese BERT-networks," in *Proc. EMNLP-IJCNLP*, 2019, pp. 3982–3992.
- [18] W. Wang et al., "MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers," *NeurIPS*, vol. 33, pp. 5776–5788, 2020.
- [19] W. Lei et al., "Re-examining the role of schema linking in text-to-SQL," in *Proc. EMNLP*, 2020, pp. 6943–6954.
- [20] V. Karpukhin et al., "Dense passage retrieval for open-domain question answering," in *Proc. EMNLP*, 2020, pp. 6769–6781.
- [21] J. Nielsen, *Usability Engineering*. Academic Press, 1993.
- [22] T. Formal, B. Piwowarski, and S. Clinchant, "SPLADE: Sparse lexical and expansion model for first stage ranking," in *Proc. SIGIR*, 2021, pp. 2288–2292.